

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 7

RELATIONAL ALGEBRA (RA)

RELATIONAL ALGEBRA: OUTLINE

Relational Query Languages

Relational Algebra Basic Operations

Relational Algebra Additional Operations

RELATIONAL QUERY LANGUAGES

- Two mathematical query languages form the basis for “real” relational query languages (e.g., **SQL**) and for implementation.

Our
focus

→ **Relational** Procedural (*step-by-step*).

Algebra

Need to describe **how** to compute a query result.

Relational
Calculus

Non-procedural (*declarative*).

Only need to state **what** query result is wanted, not how to compute it.

✎ **Relational algebra is very useful for representing and optimizing query execution plans.**

(Discussed later in the course.)

Understanding relational algebra is the key to understanding SQL and how it is processed!

RELATIONAL ALGEBRA

- The relational algebra is an algebra whose
 - **operands** are either relations or variables that represent relations.
 - **operations** perform common, basic manipulations of relations.

Closure Property

- Relational algebra is closed with respect to the relational model.
 - ✎ Each operation manipulates one or more relations and **returns a relation** as its result.

Due to the closure property, operations can be composed!

✎ A relational algebra expression is **evaluated from the inside-out**.

RELATIONAL ALGEBRA OPERATIONS

Basic Operation Symbol Action

Selection	σ	Selects tuples in a relation that satisfy a predicate
Projection	π	Removes unwanted attributes from a relation
Union	$\cup$	Finds tuples that belong to either relation 1 or relation 2 or both relation 1 and relation 2
Set difference	$-$	Finds tuples that are in relation 1 but not in relation 2
Cartesian product	$\times$	Allows the tuples in two relations to be combined

Additional operations *(not essential, but very useful)*

Intersection	$\cap$	Finds tuples that appear in both relation 1 and relation 2
Join	$\bowtie$	Cartesian product followed by a selection
Assignment	$\leftarrow$	Assigns a result to a temporary variable
Rename	ρ	Allows a relation and/or its attributes to be renamed

SELECTION: $\sigma_C(R)$

- Selects tuples (rows) that satisfy a *selection condition* C .
- The *schema* of the *result relation* is *identical* to the *schema* of the (only) *input relation*.
- A condition C has the form: **term** *op* **term** where
 - **term** is an *attribute name* or a *constant*
 - *op* is one of the comparison operators: =, $\neq$, <, $\leq$, >, $\geq$.
- Conditions can be *composed* or *negated* using *Boolean operators*.
 - $C_1 \wedge C_2$ where C_1 and C_2 are conditions and $\wedge$ means *AND*
 - $C_1 \vee C_2$ where C_1 and C_2 are conditions and $\vee$ means *OR*
 - $\neg C$ where $\neg$ means *NOT*

SELECTION: EXAMPLE

Query: Find tuples where the company is Boeing.

Plane

company	model
Airbus	A310
Airbus	A320
Airbus	A330
Airbus	A340
Boeing	B747
Boeing	B777

$\sigma_{\text{company}='Boeing'} \text{Plane}$ =

Query result

company	model
Boeing	B747
Boeing	B777

Query: Find tuples where the company is Boeing, or the model is A330.

Plane

company	model
Airbus	A310
Airbus	A320
Airbus	A330
Airbus	A340
Boeing	B747
Boeing	B777

$\sigma_{\text{company}='Boeing' \vee \text{model}='A330'} \text{Plane}$ = ?

Query result

company	model

PROJECTION: $\pi_L(R)$

- Keeps only the attributes in a *projection list* L .

✎ The **schema** of the result relation contains the **same attributes** as in the **projection list** L , with the same names that they had in the (only) input relation.

Query: Find the companies that make planes.

Plane			
company	model		
Airbus	A310	$\pi_{\text{company}} \text{Plane}$	=
Airbus	A320		
Airbus	A330		
Airbus	A340		
Boeing	B747		
Boeing	B777		

Query result

company
Airbus
Boeing

✎ The projection operator *removes duplicate tuples.*

Why?

COMPOSITION OF OPERATIONS

Recall: relational algebra operations can be **composed**.

👉 **The result of a relational algebra operation is a relation.**

Query: Find only those models made by Boeing.

Plane

company	model
Airbus	A310
Airbus	A320
Airbus	A330
Airbus	A340
Boeing	B747
Boeing	B777

$$\pi_{\text{model}}(\sigma_{\text{company}='Boeing'}\text{Plane})$$

=

Query result

model
B747
B777

Is this a correct solution?

$$\sigma_{\text{company}='Boeing'}(\pi_{\text{model}}\text{Plane})$$

SET OPERATIONS

- The set operations, each of which takes two input relations, are:

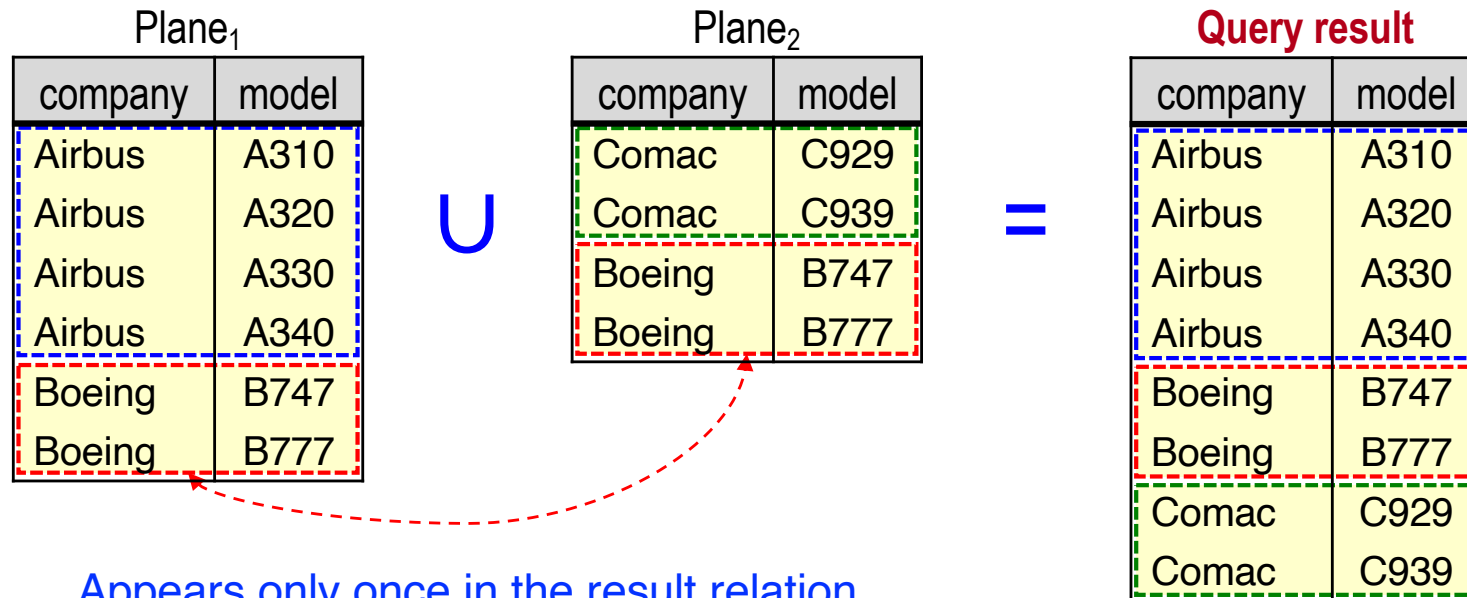
- U **union** merge tuples from both input relations
 - **set difference** remove common tuples from the first input relation
 - ∩ **intersection** keep only the tuples common to both input relations

(Intersection is not basic; it can be expressed using union and set-difference,
i.e., $r \cap s = ((r \cup s) - (r - s)) - (s - r)$).

- The two input relations must be union-compatible, which means that
 - the relations have the **same number** of attributes.
 - corresponding attributes have the **same data type**.
- The output is a single relation (**without duplicates**).

UNION: $\cup$

Query: Find tuples that appear in Plane₁, Plane₂ or both (i.e., find all the non-duplicate (unique) tuples).



Appears only once in the result relation
since set operations eliminate duplicates.

SET DIFFERENCE: -

Query: Find tuples that appear in Plane₁, but not in Plane₂.

Plane ₁			Plane ₂			Query result	
company	model		company	model		company	model
Airbus	A310	-	Comac	C929	=	Airbus	A310
Airbus	A320		Comac	C939		Airbus	A320
Airbus	A330		Boeing	B747		Airbus	A330
Airbus	A340		Boeing	B777		Airbus	A340
Boeing	B747						
Boeing	B777						

Removed from the result relation since the tuples appear in both relations.

Plane ₂			Plane ₁			Query result	
company	model		company	model		company	model
Comac	C929	-	Airbus	A310	=?		
Comac	C939		Airbus	A320			
Boeing	B747		Airbus	A330			
Boeing	B777		Airbus	A340			
			Boeing	B747			
			Boeing	B777			

CARTESIAN PRODUCT: X

- Cartesian product combines **each row** of one relation with **every row** of another relation.
- CanFly $\times$ Plane $\Rightarrow$ **72 tuples!!!**
(9 $\times$ 8)

CanFly

empNo	model
1001	B747
1001	B777
1001	A310
1002	A320
1002	A340
1002	B777
1002	C929
1003	A310
1003	C939

9 tuples

X

Plane

company	model
Airbus	A310
Airbus	A320
Airbus	A330
Airbus	A340
Boeing	B747
Boeing	B777
Comac	C929
Comac	C939

8 tuples

=

Query result

empNo	model	company	model
1001	B747	Airbus	A310
1001	B747	Airbus	A320
1001	B747	Airbus	A330
1001	B747	Airbus	A340
1001	B747	Boeing	B747
1001	B747	Boeing	B777
1001	B747	Comac	C929
1001	B747	Comac	C939
1001	B777	Airbus	A310
1001	B777	Airbus	A320
1001	B777	Airbus	A330
1001	B777	Airbus	A340
1001	B777	Boeing	B747
1001	B777	Boeing	B777
1001	B777	Comac	C929
1001	B777	Comac	C939
1001	A310	Airbus	A310
1001	A310	Airbus	A320
⋮	⋮	⋮	⋮

72 tuples



INTERSECTION: $\cap$

Query: Find tuples that appear in both Plane₁ and Plane₂.

Plane ₁		$\cap$	Plane ₂		=	Query result	
company	model		company	model		company	model
Airbus	A310	$\cap$	Comac	C929	=	Boeing	B747
Airbus	A320		Comac	C939		Boeing	B777
Airbus	A330		Boeing	B747			
Airbus	A340		Boeing	B777			
Boeing	B747						
Boeing	B777						

Plane ₂		$\cap$	Plane ₁		= ?	Query result	
company	model		company	model		company	model
Comac	C929	$\cap$	Airbus	A310	= ?		
Comac	C939		Airbus	A320			
Boeing	B747		Airbus	A330			
Boeing	B777		Airbus	A340			
			Boeing	B747			
			Boeing	B777			

JOIN: ⋈

- Generating all possible tuple combinations of two relations is usually not meaningful.

Example: For the relations *CanFly* and *Plane*, combining each *CanFly* and *Plane* tuple having a matching *model* value is more meaningful than *CanFly* $\times$ *Plane*.

- A *join* is a **Cartesian product** followed by a **selection**:

$$R_1 \bowtie_c R_2 = \sigma_c(R_1 \times R_2) \quad \text{or} \quad R_1 \text{ JOIN}_c R_2 = \sigma_c(R_1 \times R_2)$$

- Types of joins:

θ -join Allows arbitrary conditions in the selection.

equijoin All conditions are equality.

natural join Combines two relations based on the equality of the attribute values with the same names.

👉 **Both equijoin and natural join project the result on only one set of the common join attributes.**

CanFly

empNo	model
1001	B747
1001	B777
1001	A310
1002	A320
1002	A340
1002	B777
1002	C929
1003	A310
1003	C939

Plane

company	model
Airbus	A310
Airbus	A320
Airbus	A330
Airbus	A340
Boeing	B747
Boeing	B777
Comac	C929
Comac	C939

JOIN: θ -JOIN

θ -join allows arbitrary conditions in the selection.

Example: Join a relation with itself (*self-join*) using the condition:

$c = \text{Flight1.destination} = \text{Flight2.origin} \wedge \text{Flight1.arrivalTime} < \text{Flight2.departureTime}$

(Note that in this example an attribute's name needs to be qualified with the relation's name in the selection condition to disambiguate it since both input relations have the same attribute names.)

What should we get
(i.e., how would you describe the flights)?

Flight1

flight#	origin	destination	departure Time	arrival Time
334	HKG	PVG	12:00	14:14
335	PVG	HKG	15:00	17:14
336	HKG	PVG	18:00	20:14
337	PVG	HKG	20:30	23:53
394	PEK	PVG	19:00	21:30
395	PVG	PEK	21:00	23:43



Flight2

flight#	origin	destination	departure Time	arrival Time
334	HKG	PVG	12:00	14:14
335	PVG	HKG	15:00	17:14
336	HKG	PVG	18:00	20:14
337	PVG	HKG	20:30	23:53
394	PEK	PVG	19:00	21:30
395	PVG	PEK	21:00	23:43

JOIN: θ -JOIN (CONT'D)

θ -join allows arbitrary conditions in the selection.

Flight1 $\bowtie$ Flight1.destination=Flight2.origin $\wedge$ Flight1.arrivalTime<Flight2.departureTime Flight2

Flight1. Flight#	Flight1. Origin	Flight1. Destination	Flight1. Departure Time	Flight1. Arrival Time	Flight2. Flight#	Flight2. Origin	Flight2. Destination	Flight2. Departure Time	Flight2. Arrival Time
334	HKG	PVG	12:00	14:14	335	PVG	HKG	15:00	17:14
335	PVG	HKG	15:00	17:14	336	HKG	PVG	18:00	20:14
336	HKG	PVG	18:00	20:14	337	PVG	HKG	20:30	23:53
334	HKG	PVG	12:00	14:14	337	PVG	HKG	20:30	23:53
336	HKG	PVG	18:00	20:14	395	PVG	PEK	21:00	23:43
334	HKG	PVG	12:00	14:14	395	PVG	PEK	21:00	23:43

What is the result if we add the condition: ... $\wedge$ Flight1.origin<>Flight2.destination?

JOIN: NATURAL JOIN

Natural join combines two relations based on the equality of the values of attributes with the same names.

CanFly $\bowtie_n$ Plane $\Leftrightarrow$ CanFly $\bowtie$ Plane

$n \Rightarrow$ look for attributes with common names in the two relations.

CanFly JOIN_n Plane $\Leftrightarrow$ CanFly JOIN Plane

CanFly JOIN_{model} Plane

Cartesian product $\Rightarrow$ 72 tuples; join $\Rightarrow$ 9 tuples.

CanFly	
empNo	model
1001	B747
1001	B777
1001	A310
1002	A320
1002	A340
1002	B777
1002	C929
1003	A310
1003	C939

9 tuples

$\bowtie_n$

Plane	
company	model
Airbus	A310
Airbus	A320
Airbus	A330
Airbus	A340
Boeing	B747
Boeing	B777
Comac	C929
Comac	C939

8 tuples

=

Query result		
empNo	model	company
1001	B747	Boeing
1001	B777	Boeing
1001	A310	Airbus
1002	A320	Airbus
1002	A340	Airbus
1002	B777	Boeing
1002	C929	Comac
1003	A310	Airbus
1003	C939	Comac

9 tuples

 Cannot qualify the common join attributes in a natural join. Why?

OUTER JOIN

- An **extension** of the **natural join operation** that avoids loss of information.
- Computes the natural join and then **adds tuples** from one input relation that **do not have matching tuples** in the other input relation to the join result.
- Uses **null** values to fill in missing information.
 - Recall that **null** signifies that the value is unknown or does not exist.

 **All comparisons involving null are false.**
(In particular, **null=null** is false.)

OUTER JOIN (CONTD)

- Natural join returns only the tuples that match on the join attributes (i.e., the “good tuples”).

Loan				Borrower			Query result			
loan Number	amount	branch Name		client Name	loan Number		loan Number	amount	branch Name	client Name
L-170	30000	Central	⋈	Pat Lee	L-170	=	L-170	30000	Central	Pat Lee
L-260	170000	Tsim Sha Tsui		Mary Kwan	L-230		L-230	40000	Central	Mary Kwan
L-230	40000	Central		Ted Hayes	L-155					

- The fact that
 - loan L-260 in the Loan relation has no borrower (i.e., no matching loanNumber) in the Borrower relation is not shown in the result;
 - customer Ted Hayes in the Borrower relation holds loan L-155 that does not appear in the Loan relation is also not shown in the result.

👉 Both tuples have no matching tuple in the other relation.

LEFT OUTER JOIN:

Adds to the natural join query result **all tuples in the left input relation** (Loan) **that do not match** with any tuple in the right input relation (Borrower) and fills in **null** for the missing information.

Loan			Borrower		Query result			
loan Number	amount	branch Name	client Name	loan Number	loan Number	amount	branch Name	client Name
L-170	30000	Central	Pat Lee	L-170	L-170	30000	Central	Pat Lee
L-260	170000	Tsim Sha Tsui	Mary Kwan	L-230	L-230	40000	Central	Mary Kwan
L-230	40000	Central	Ted Hayes	L-155	L-260	170000	Tsim Sha Tsui	null

 The join result now shows that loan L-260 has no borrower (i.e., has no client).

RIGHT OUTER JOIN:

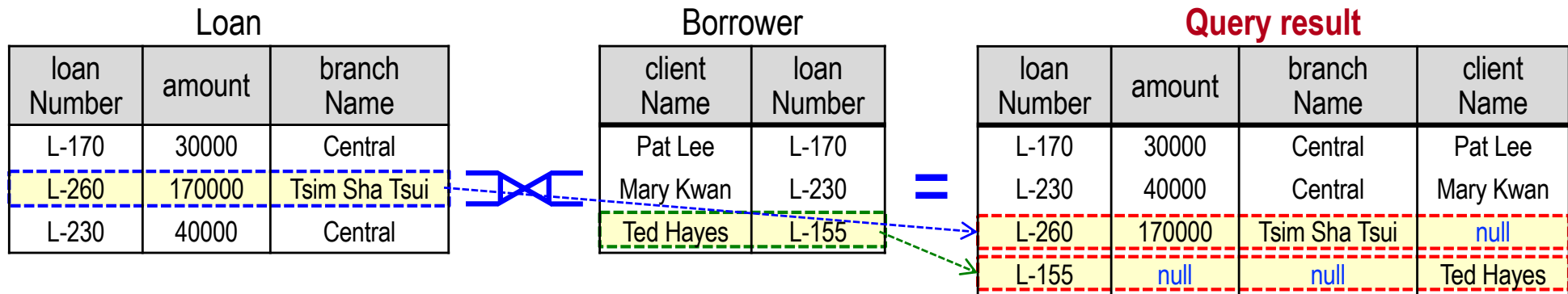
Adds to the natural join result relation **all tuples in the right input relation** (**Borrower**) **that do not match** with any tuple in the left input relation (**Loan**) and fills in **null** for the missing information.

Loan			Borrower		Query result			
loan Number	amount	branch Name	client Name	loan Number	loan Number	amount	branch Name	client Name
L-170	30000	Central	Pat Lee	L-170	L-170	30000	Central	Pat Lee
L-260	170000	Tsim Sha Tsui	Mary Kwan	L-230	L-230	40000	Central	Mary Kwan
L-230	40000	Central	Ted Hayes	L-155	L-155	null	null	Ted Hayes

 The join result now shows that loan L-155 is a nonexistent loan (i.e., has no amount and no branch name).

FULL OUTER JOIN:

Adds to the natural join result relation **all tuples in both input relations that did not match** with any tuples in the other input relation and fills in **null** for missing information.



👉 The join result now shows both that

- loan L-260 has no borrower (i.e., has no client).
- loan L-155 is nonexistent (i.e., has no amount and no branch).

RENAMING: ρ

- **Assigns a name** to, or **renames** the attributes in, a relational-algebra expression.

$\rho_x(E)$ assigns name x to the result of E (E can be a relation name or a relational algebra expression)

$\rho_{x(A_1, A_2, \dots, A_n)}(E)$ assigns name x to the result of E *and* renames the attributes of E as $A_1, A_2, \dots, A_n$

 **Renaming is necessary when taking the Cartesian product or join of a relation with itself.**

Example

$\rho_{\text{Flight1}}(\text{Flight}) \bowtie_{\text{Flight1.destination}=\text{Flight2.origin} \wedge \text{Flight1.arrivalTime} < \text{Flight2.departureTime}} \rho_{\text{Flight2}}(\text{Flight})$

RELATIONAL ALGEBRA: SUMMARY

- Relational algebra defines a set of algebraic operations that operate on relations and output a relation as their result.
- The operations can be composed to express queries.
- The operations can be divided into:
 - basic operations
 - σ selection
 - π projection
 - $\cup$ union
 - $-$ set difference
 - $\times$ Cartesian product.
 - additional operations that either
 - can be expressed in terms of the basic operations or
 - add further expressive power to the relational algebra.